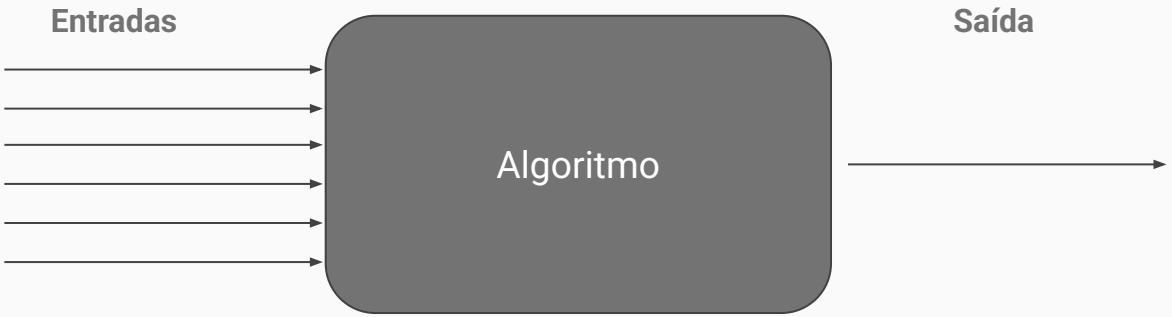
Introdução à programação - 2
Lógica de Programação

Curso Microcontroladores

Prof. Luiz Marangoni - 2026

Ciência da Computação

Resolução de problemas através de um software/código computacional



Algoritmo

1. Sequência finita de ações executáveis que visam obter uma solução para um determinado tipo de problema

“É uma receita que mostra passo a passo os procedimentos necessários para a realização de uma tarefa.”

https://coodesh.com/blog/dicionario/o-que-e-algoritmo/

Lógica de programação

Conceitos

Raciocínio -> forma como os computadores/sistemas/máquinas resolvem problemas

Conceitos

- Variáveis e Constantes
- Estruturas de dados e coleções
- Tipos de variáveis
- Operadores Matemáticos
- Operadores Lógicos e comparativos
- Estruturas de repetição
- Estruturas Condicionais

Linguagens programação implementam estes conceitos

Lógica de programação

Conceitos

Variáveis e Constantes

Espaços reservados na memória para guardar valores que podem variar e/ou serem utilizados várias vezes no programa - Nome (endereços destes espaços na memória)

Tipos de variáveis

Quantidade de espaço na memória que deverá ser reservado para aquela informação

- Inteiro
- float (ponto flutuante)
- String
- Boolean
- ...

Estruturas de dados e coleções

Listas - Relacionamento de dados de um mesmo tipo e sua organização na memória

Dicionários - Relacionamento de dados de diferentes tipos e sua organização na memória

Operadores Lógicos e comparativos

Estruturas de repetição

Operadores Matemáticos

Estruturas Condicionais

Lógica de programação

Conceitos

Operadores Matemáticos

Formas de relacionamento dos dados no programa seguindo operações matemáticas: +, -, *, /, sqrt,...

Operadores Lógicos e comparativos

Formas de relacionamento dos dados no programa seguindo operações lógicas - E / OU / NÃO

Comparativos - >, <, >=, <=,

Estruturas de repetição

Repetição - por um número determinado de vezes (For)

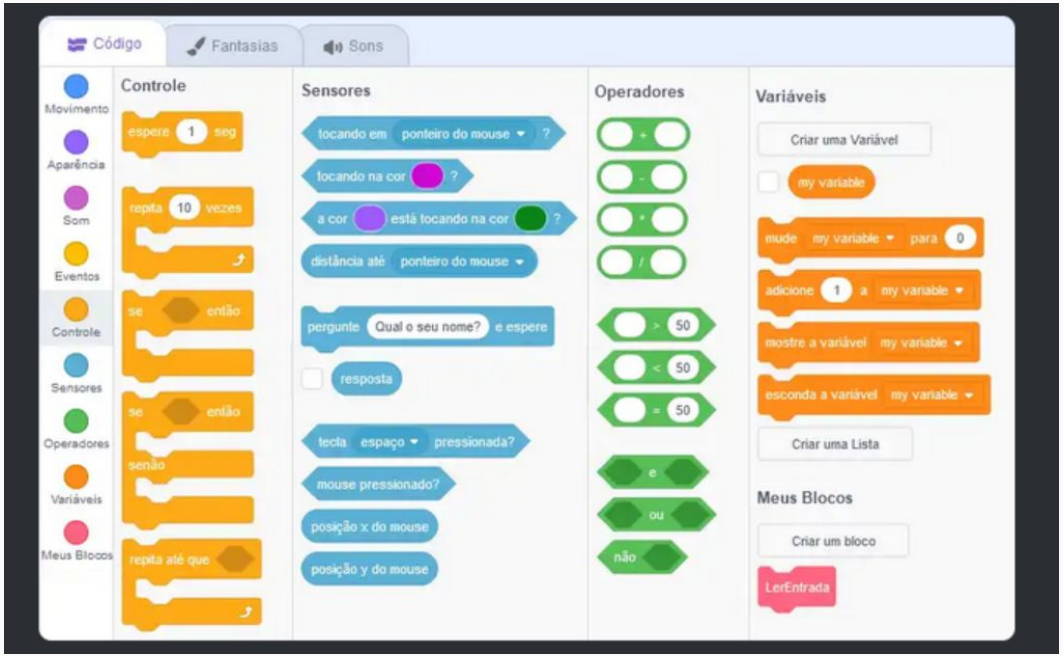
Repetição enquanto uma condição for válida (while)

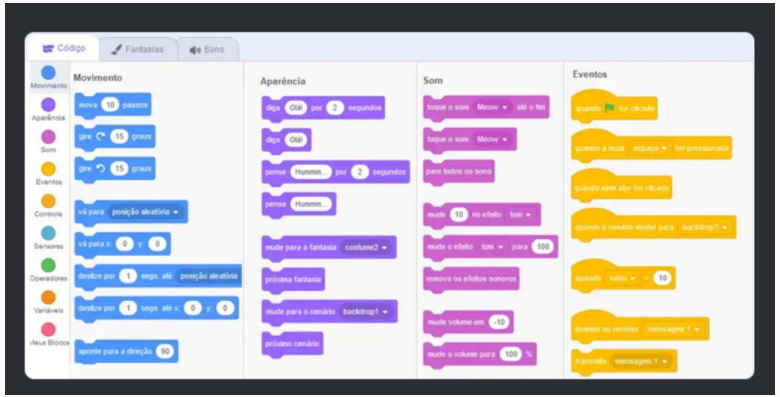
Estruturas Condicionais

Teste de condições - if/then/else, else if, case

Lógica de programação

Aplicação na linguagem gráfica - Scratch





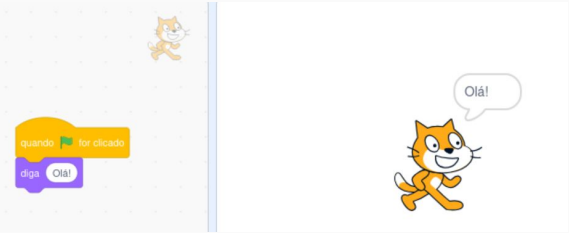
Lógica de programação

Aplicação na linguagem gráfica - Scratch

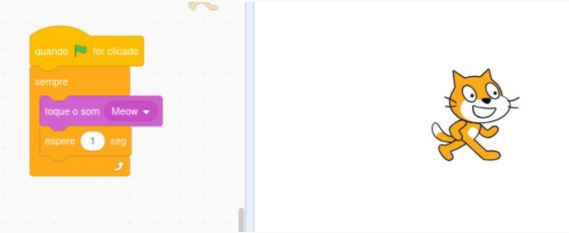
Aplicação na linguagem gráfica - Scratch

Demonstração

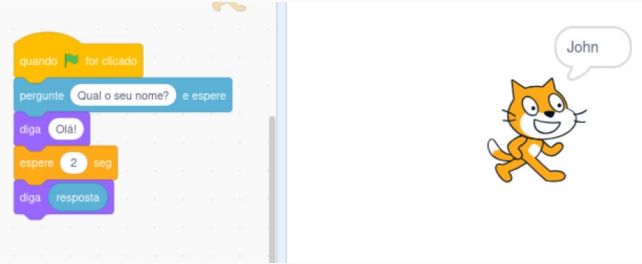
Demonstrações Scratch



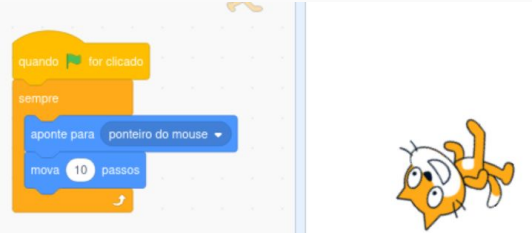
1. Hello world



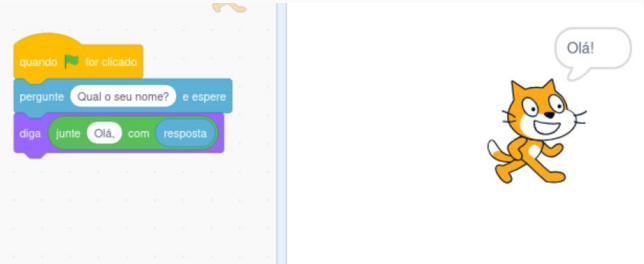
4. Fazer o gato miar continuamente em intervalo de 1s - **estrutura de repetição**



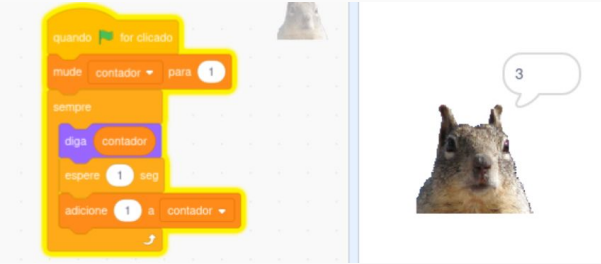
2. Perguntar e esperar a resposta



5. Scratch segue o mouse

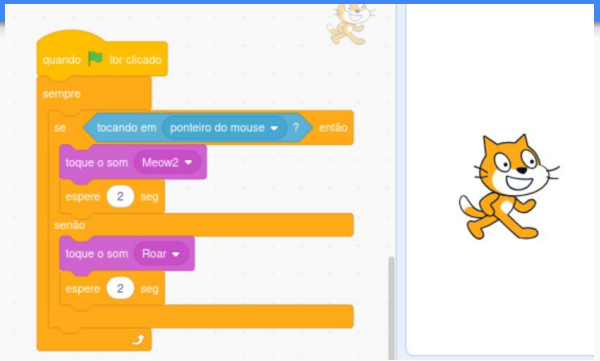


3.Operador “join”

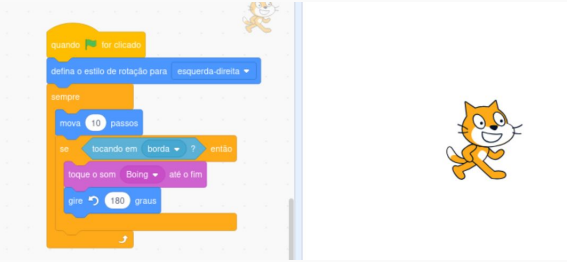


6. Capivara que conta - Uso de variáveis

Demonstrações Scratch

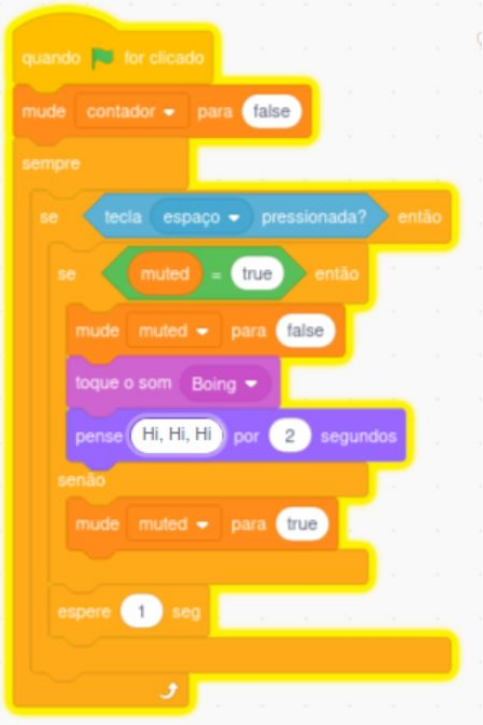


7. Estrutura condicional - If/then



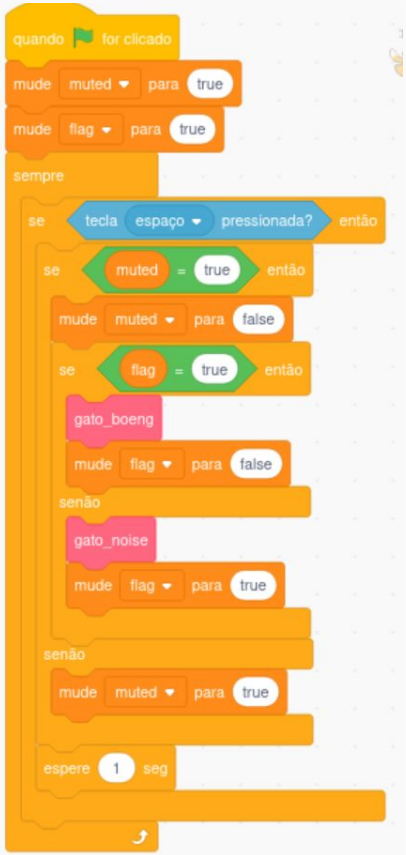
8. Detectando colisão com as bordas (eventos)

9. Verifica se a tecla de espaço está pressionada



10. Ao pressionar o botão de espaço, o programa verifica a variável “muted”, se for verdadeira, o gato irá fazer um ruído e pensar por 2 segundos

Demonstrações Scratch



11. Criação de funções - abstrações

Demonstrações - Sistemas embarcados

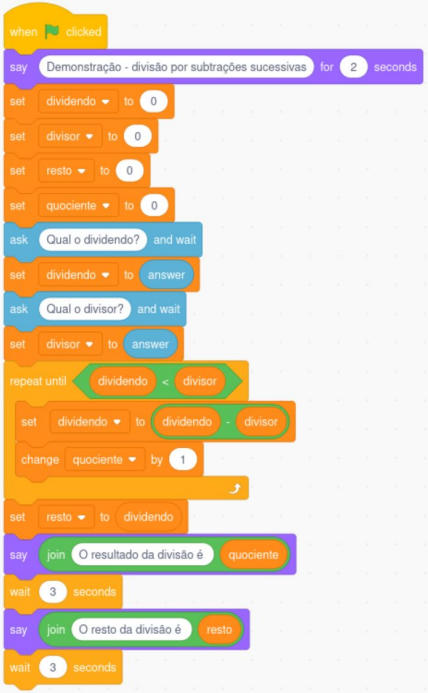
1. Exemplo - Divisão de inteiros por subtrações sucessivas

Dividir o número X pelo número Y (ambos inteiros) através de subtrações sucessivas (método utilizado em microcontroladores RISC)

Algoritmo:

1. Solicite o número X - dividendo
2. Solicite o número Y - divisor
3. Crie uma variável quociente e inicie com 0
4. Crie uma variável resto e inicie com 0
5. Repita até dividendo < divisor
   a. dividendo = dividendo - divisor
   b. quociente = quociente + 1
6. resto = dividendo
7. resultado = quociente
8. Informe o resultado e o resto da operação

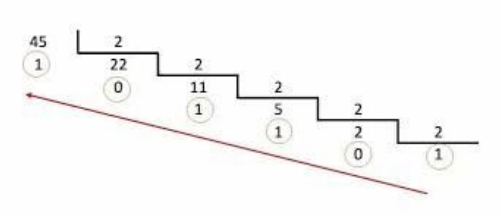
[Programa]



2. Exemplo - Conversão de um número de decimal para binário

Converter um número decimal inteiro (8 bits < 255) e exibir o resultado

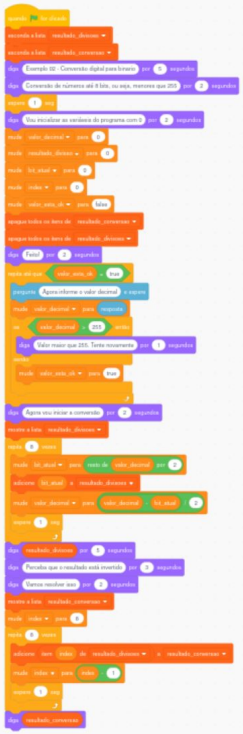
Para realizar a conversão, divida o número sucessivamente por 2 e guarde o resto (0 ou 1) enquanto a divisão resultar num número maior que 1 (número inteiro). O resultado será a sequência de zeros/uns armazenados.



Algoritmo:

1. Solicite o número - valor_decimal
2. Verifique se o número é menor ou igual a 255 (limite de 8 bits)
   Se não:
   informe o erro
   solicite novamente o número

   Se sim:
   Repita 8 vezes (8 bits)
   divida o valor_decimal X por 2
   guarde o resto da divisão (bit_atual) numa lista
   novo número X = (X - bit_atual)/2 (evita erros de arredondamento)
3. Exiba o resultado
4. Inverta a ordem da lista
5. Exiba o resultado correto



Programa

2. Exemplo - Conversão de um número de decimal para binário

Converter um número decimal inteiro (8 bits < 255) e exibir o resultado - Usando funções

Algoritmo:

1. Apresente o programa ao usuario
2. Inicialize as variaveis
3. Solicite o número - valor_decimal e verifique se menor que 255
4. Converta o número para binario
5. Exiba o resultado - diga que está invertido
6. Inverta a ordem da lista
7. Exiba o resultado correto

programa

Atividade para hoje

Implementar no Scratch os algoritmos desenvolvidos para:

1) Divisão por subtrações sucessivas
2) Multiplicação por somas sucessivas
3) Conversão de um número decimal para binário

2. Multiplicação por somas sucessivas

Realizar a operação de multiplicação por somas sucessivas

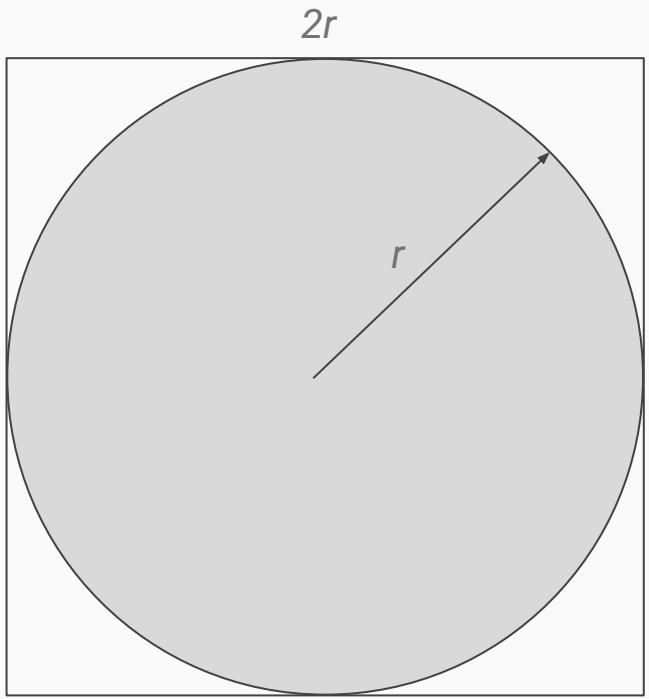
Algoritmo:

1. Apresente o programa ao usuario
2. Inicialize as variaveis
3. Solicite o 1o fator
4. Solicite o multiplicador
5. Repita o número de vezes do multiplicador
   a. produto = produto + 1o_fator
6. Exiba o resultado

programa

Desafio - Cálculo do valor de pi pelo método de Monte Carlo (probabilístico)

3. Fundamento do método - Monte Carlo



Quadrado de lado L = 2r

Área do quadrado: Aquad=4r2

Círculo de lado r

Área do Círculo: Acirc=πr2

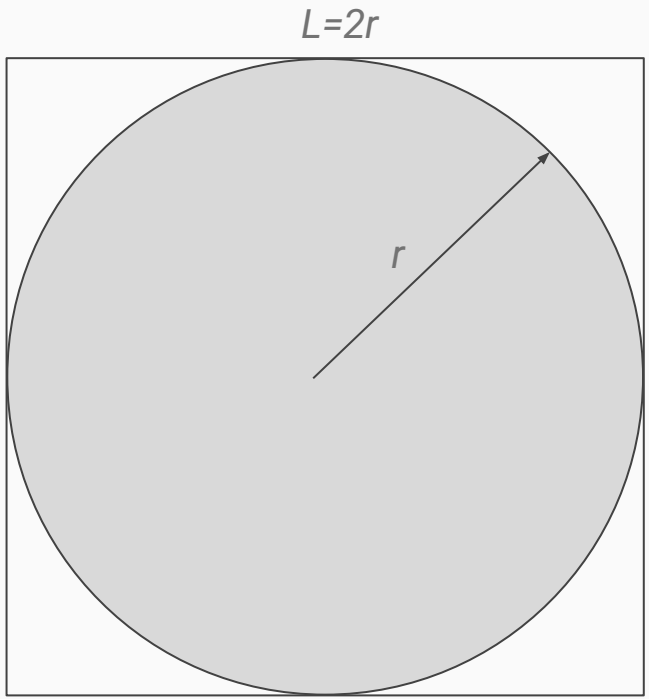
Relação entre as áreas

Aquad/Acirc = 4r2 / πr2 = 4/π

Cálculo de pi:

π ≈ 4 * Aquad/Acirc

3. Fundamento do método - Monte Carlo



A Razão Probabilística

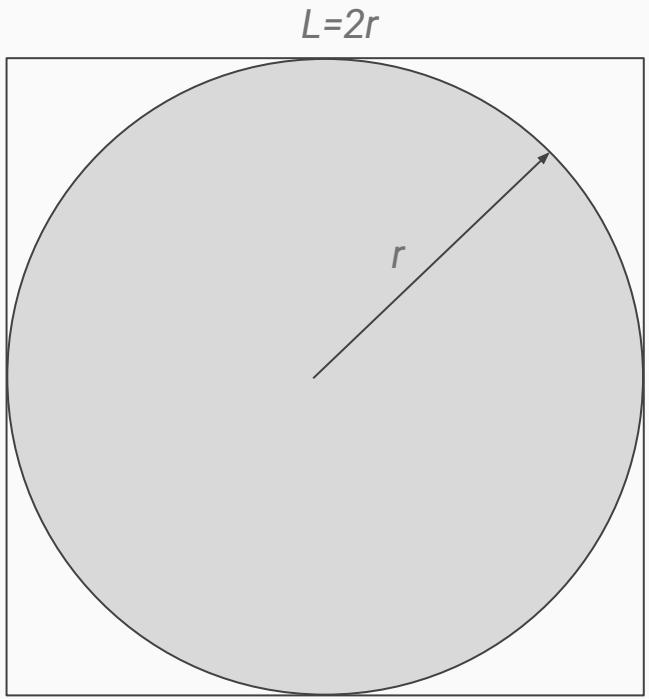
Se dispararmos "dardos" (pontos aleatórios) uniformemente sobre a área do quadrado, a probabilidade **P de um dardo cair dentro do círculo é a razão entre a área do círculo e a área do quadrado:**

$$P = \frac{A_c}{A_q} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

Dessa fórmula, isolamos o π:

$$\pi = 4 \times P$$

3. Fundamento do método - Monte carlo



3. O Algoritmo (Simulação)

Para estimar essa probabilidade P no Scratch ou em qualquer linguagem, seguimos estes passos:

1. **Gerar N pontos:** Sorteie valores aleatórios para x e y entre $-r$ e r (ou 0 e 1).
2. **Verificar a condição:** Um ponto (x, y) está dentro do círculo se a sua distância até a origem for menor ou igual ao raio. Usamos o Teorema de Pitágoras:
   - Condição: $x^2 + y^2 \leq r^2$
3. **Contabilizar:** Mantenha um contador para o `total_de_pontos` e outro para `pontos_no_circulo`.
4. **Calcular:** No final de milhares de iterações, aplique a fórmula:

$$\pi \approx 4 \times \frac{\text{pontos_no_circulo}}{\text{total_de_pontos}}$$

Implementar no scratch, em duplas.
Utilizando o programa referência da próxima página

3. Solução

Solução 1: Sem exibição na tela dos pontos

3. O Algoritmo (Simulação)

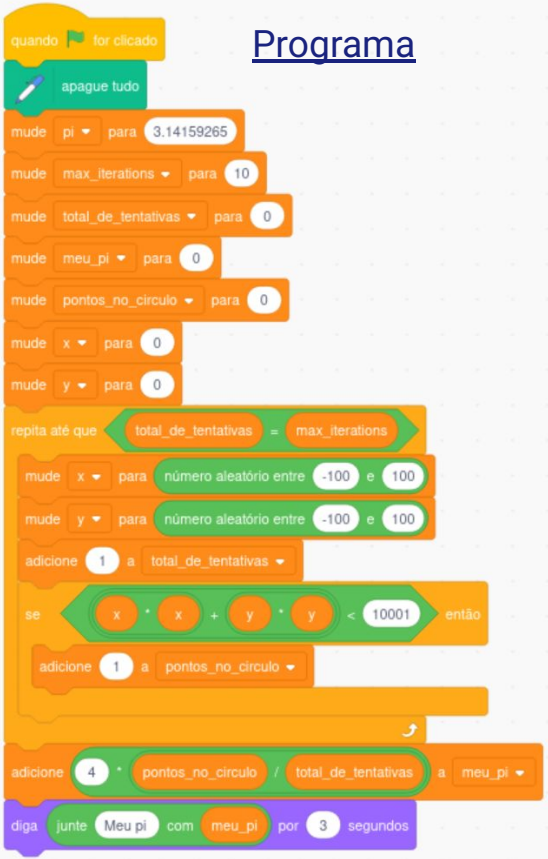
Para estimar essa probabilidade P no Scratch ou em qualquer linguagem, seguimos estes passos:

1. **Gerar N pontos:** Sorteie valores aleatórios para x e y entre $-r$ e r (ou 0 e 1).
2. **Verificar a condição:** Um ponto (x, y) está dentro do círculo se a sua distância até a origem for menor ou igual ao raio. Usamos o Teorema de Pitágoras:
   - Condição: $x^2 + y^2 \leq r^2$
3. **Contabilizar:** Mantenha um contador para o `total_de_pontos` e outro para `pontos_no_circulo`.
4. **Calcular:** No final de milhares de iterações, aplique a fórmula:

$$\pi \approx 4 \times \frac{\text{pontos_no_circulo}}{\text{total_de_pontos}}$$

Testar o programa com 10, 100, 1000, 10000 amostras. O que ocorre com o resultado?

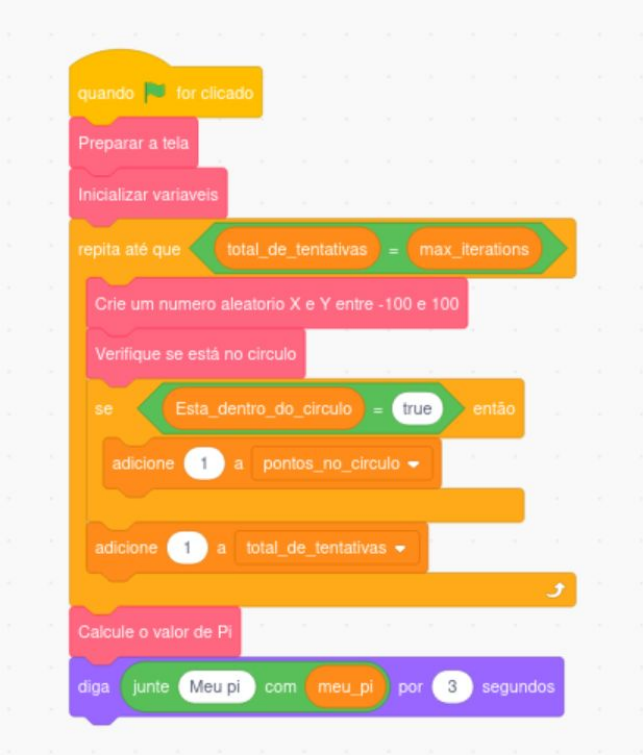
Programa



3. Solução

Solução 2: Utilizando funções

[Programa]



3. Solução

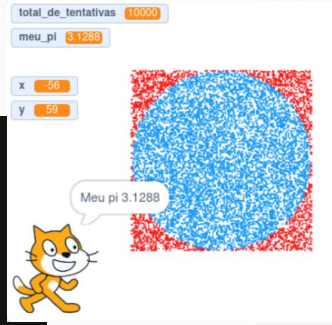
Solução 3: Exibição na tela dos pontos

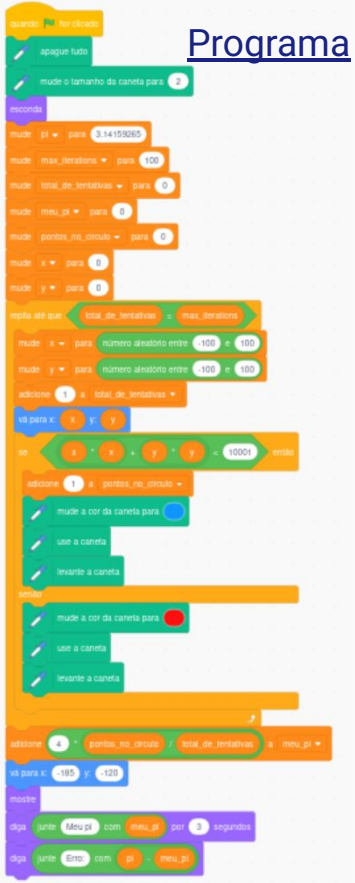
3. O Algoritmo (Simulação)

Para estimar essa probabilidade P no Scratch ou em qualquer linguagem, seguimos estes passos:

1. **Gerar N pontos:** Sorteie valores aleatórios para x e y entre $-r$ e r (ou 0 e 1).
2. **Verificar a condição:** Um ponto (x, y) está dentro do círculo se a sua distância até a origem for menor ou igual ao raio. Usamos o Teorema de Pitágoras:
   - Condição: $x^2 + y^2 \leq r^2$
3. **Contabilizar:** Mantenha um contador para o `total_de_pontos` e outro para `pontos_no_circulo`.
4. **Calcular:** No final de milhares de iterações, aplique a fórmula:

$$\pi \approx 4 \times \frac{\text{pontos_no_circulo}}{\text{total_de_pontos}}$$





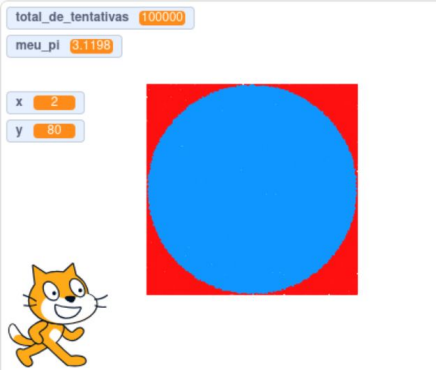
Programa

Testar o programa com 10, 100, 1000, 10000 amostras. O que ocorre com o resultado?

3. Solução

Solução 1: link

Solução 2: link



Atividade aula - Cálculo do valor de Pi através de séries infinitas

Fundamento do método - Série de Leibniz

- **A Lógica:** A série de Leibniz estabelece que:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} \cdots$$

Crie um programa no scratch que

1. Mantenha uma variável `denominador` que começa em 1 e aumenta de 2 em 2 a cada ciclo.
2. Alterne o sinal da operação (adição e subtração) a cada iteração.
3. Acumule o resultado em uma variável `soma` e multiplique por 4 ao final.

Conceito Superior: Cálculo Diferencial e Integral, **Convergência de Séries Numéricas** e **Análise de Erro de Truncamento**.

Fundamento do método - Série de Leibniz

- **A Lógica:** A série de Leibniz estabelece que:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} \cdots$$

Algoritmo sugerido:

1. **quando bandeira verde for clicada**
2. **apague tudo**
3. **defina pi_real para 3.14159265**
4. **defina soma para 0**
5. **defina denominador para 1**
6. **defina sinal para 1**
7. **defina iteracoes para 0**
8. **defina max_iteracoes para 1000**
9. **repita até <iteracoes = max_iteracoes>**
    - defina **soma** para `(soma + (sinal * (1 / denominador)))`
    - mude **denominador** por `2`
    - defina **sinal** para `(sinal * -1)`
    - mude **iteracoes** por `1`
10. **defina meu_pi para (4 * soma)**
11. **defina erro para (pi_real - meu_pi)**
12. **diga (junte [Meu pi =] (meu_pi)) por 3 segundos**
13. **diga (junte [Erro =] (erro))**